Backend and Runtime

CS 1622

Jarrett Billingsley

Class Announcements

- today is kind of an intro to the second half of the course
- how was the exaaaaaaaaam?
- also I'm realizing that there are actually *way* more minutes of lecture in the summer term than in spring/fall so I shouldn't feel pressured to make every lecture take all 105 minutes

Welcome to the Backend!

Through the looking glass

- a compiler translates from a source language to a target language.
  - well, now we're **done with the source language.**
- once the AST has been constructed and **semantically analyzed...**
  - we have a "correct" program!
- from here on out, we're assuming that we're working with **an AST that represents a correct program.**
  - later we'll come back and look at **intermediate representation** (IR), which lets us abstract away even the AST itself.

What is the backend responsible for?

- a compiler translates from the **source** language to the **target.**
- so far, all we've done is *validate* the source code.
- that means the backend **does the actual translation!**

it maps each part of the source language into some target language instructions that do **the same thing** as what the programmer wrote.

```
f();          ➡   jal  f

x = 5;        ➡   li   t0, 5
                  sw   t0, x

f(x + 3);     ➡   lw   a0, x
                  add  a0, a0, 3
                  jal  f
```

Correctness

- the goal of an HLL is to provide a **solid base** of abstractions on which you can build software *better than you could do in assembly*.
- **above all,** a compiler's number one priority is to produce a **correct** translation of the source code.
  - if a compiler *mistranslates code,* you've lost all the supposed guarantees that the HLL provides.
- **bugs in compilers can be incredibly serious.**
  - a compiler that fails to catch certain mistakes, or which produces incorrect code, can produce executables which leave *millions of computers* vulnerable to attacks, in the worst cases.
- so, what does "correct" code look like?

It doesn't have to look pretty

- think back to 447. I want to do **x++** for some global variable **x**.

```
# x++
lw   t0, x
addi t0, t0, 1
sw   t0, x
```

```
add_ints:
    add  v0, a0, a1
    jr   ra

main:
    # x++
    lw   a0, x
    li   a1, 1
    jal  add_ints
    sw   v0, x
```

```
# push 1
li   t0, 1
addi sp, sp, -4
sw   t0, 0(sp)

# push x
lw   t0, x
addi sp, sp, -4
sw   t0, 0(sp)

# pop 2, add, push sum
lw   t0, 0(sp)
lw   t1, 4(sp)
addi sp, sp, 8
add  t0, t0, t1
addi sp, sp, -4
sw   t0, 0(sp)

# pop x
lw   t0, 0(sp)
addi sp, sp, 4
sw   t0, x
```

these all have the **same effect:** x's value is incremented by 1. so, they are all **correct.**

what varies between them is the **code quality.**

Output code quality

- the code our compiler outputs can be measured in a few ways:
  - **speed:** how fast the output code runs
    - in the same span of time, a fast program can do more useful work than a slow program can.
  - **size:** how many bytes the output code takes up
    - if e.g. your target is a microcontroller with 4 KiB of ROM…
    - your output code has to be as small as possible to fit.
- despite the last slide, **smaller** code is *not necessarily* faster code…
  - so compilers often give you the choice of which to **optimize for.**
  - but optimizing code to be faster or smaller is an *optional* task, and is not required for correct output.

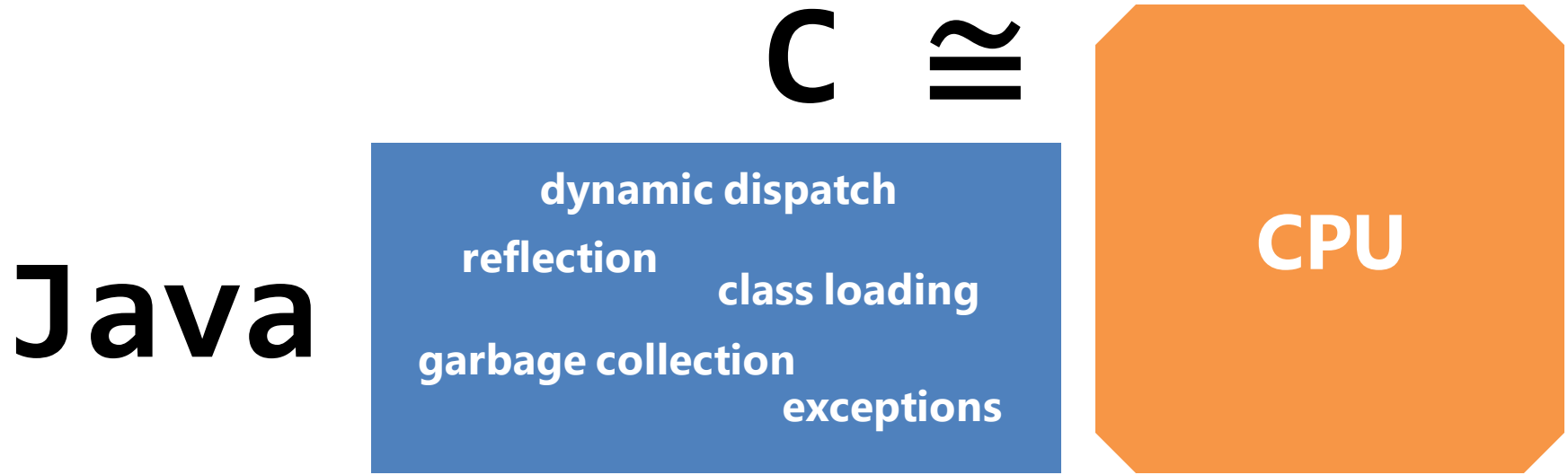
Goals and non-goals

- in the old days, the code quality of most compilers was… lacking.
  - much more like the third example on that slide than like the first.
- higher code quality requires **a more sophisticated backend.**
  - these algorithms were impractical on the slow computers those early compilers ran on, or literally *hadn't been invented yet*.
- but more complex algorithms means **a higher chance of messing them up** and making a compiler that produces **incorrect code!**
- so **our goal in this class** will be to make a ***correct*** compiler.
  - the code it produces may be *big, slow, and ugly,* but it's okay for teaching purposes.
- towards the end we'll start to look at improving code quality…
  - but you won't be implementing that. it's some *heavy* stuff.

Runtime

From compile-time to runtime

- the compiler is just the *first part* of enforcing the HLL's abstractions.
- depending on how well the **source language's semantics match the target's,** we can have a little or a **lot** to do at runtime.



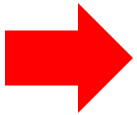
Does that mean we need a VM?

- well, not necessarily…
- there can be many **source language features which the target language does not support,** which can be even small things.
  - think about it: does MIPS have an **if-else** instruction?
    - *no*. so those have to be built out of simpler instructions.
- if the target language is a CPU, chances are all it can do is:
  - move numbers around
  - do arithmetic and logical operations on numbers
  - choose which steps to go to
  - uhhhhhhhhhhhhhhhhhh **that's it!**
- so there are like, three parts to this?
  - the **code generation**
  - the **ABI (application binary interface)**
  - and the **runtime library**

Code generation (codegen)

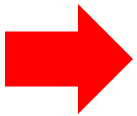
- we saw this: **codegen** is where each operation in the source language is mapped to the target language.

```
x = y
```



```
lw   t0, y
sw   t0, x
```

```
f(x)
```



```
lw   a0, x
jal  f
```

```
while(x != 10)
```



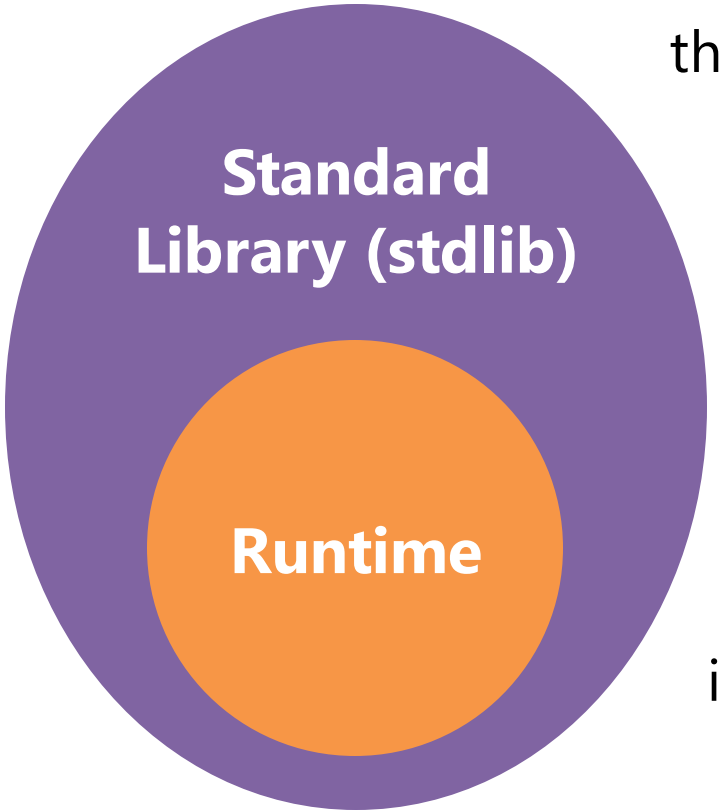
```
_top:
  lw   t0, x
  beq  t0, 10, _end
```

The ABI

- most programs run on computers with **operating systems.**
  - even those that don't, have to interact with the hardware and *themselves* (e.g. one function calling another).
- the **Application Binary Interface (ABI)** defines several things:
  - the **target language** – which in our case is a CPU ISA
  - the **calling convention(s),** which dictate how function calls work
  - the way values are **represented in memory**
  - how **system calls** work, for interacting with the OS
  - where things are **located in memory** (stack, heap, globals, etc.)
  - how the code is **packaged into an executable file**
  - and much more!

The runtime library

- also called "the runtime," confusingly
- it comes with your language; either statically or dynamically linked
  - for real why is 449 *not* a prereq for this course



the **standard library (stdlib)** has a set of useful, but *non-critical* functionality.

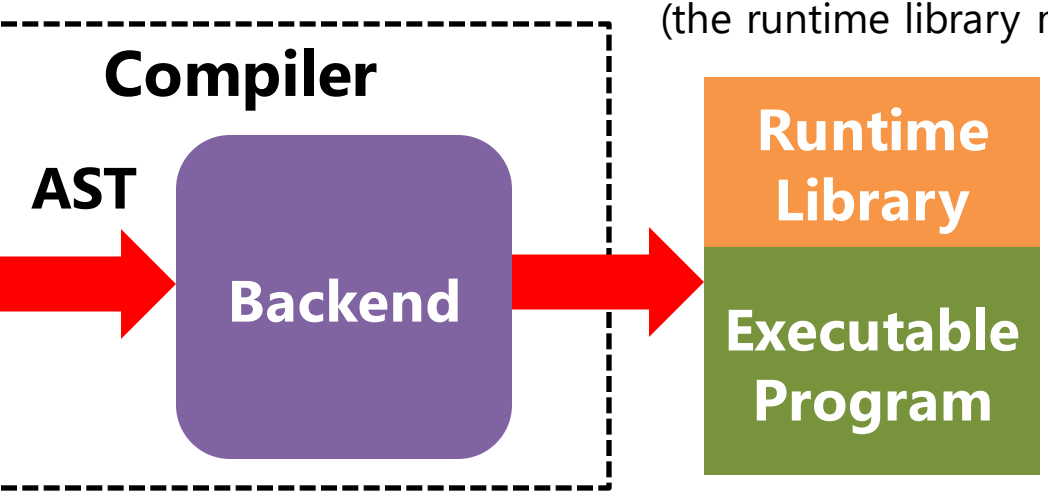
the **runtime library** is *essential* to make *language* features work at run-time.

in Rust, the runtime is called "**core**", and the standard library is "**std**".

in a lot of languages, the line is... blurrier.

Putting it all together

- to sum up:



when executed, the **runtime library** handles the source-language features which **don't exist on the target.**

the backend generates machine code and puts it in an executable which **conforms to the ABI.**

uhh… so how is this code generated, anyway?

Codegen

How do you eat an elephant?

- **codegen** seems like a giant problem with no easy place to start.
- but *any* problem can be broken down into smaller ones.

a lot of it is **filling in templates,** like mad libs for code.

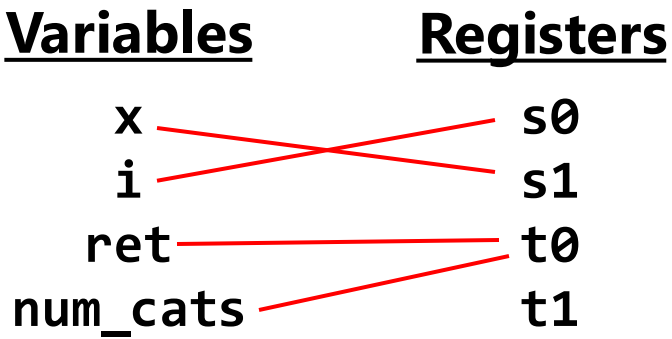
```
while cond {
  code
}
```



```
_top:
  b__ __, __, _end

  code
_end:
```

other parts are **allocation,** deciding what lives where, and when.



just keep in mind that our goal is to generate *correct* code, not *great* code.

Getting a flavor of it: codegen for a function

- as you (hopefully) learned before, every function gets a **stack frame.**
  - this is where it stores saved registers and local variables.

the template for a function's code is something like…

```
name:
    set up stack

    function body

    clean up stack
    return
```

how **big** does the stack frame have to be?

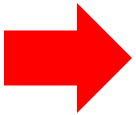
```
fn main() {
  let x = 0;
  for i in 0, 10 {
    x += i;
  }
  let y = g(x);
  println(y);
}
```

the **symbol table** can tell us how many local variables we have.

Codegen for expressions

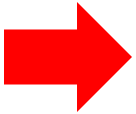
- expressions calculate **values,** and in a CPU values go in **registers.**
- an expression in *isolation* doesn't really tell you what to do though.

```
f(16)
```



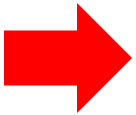
```
li  a0, 16
jal f
```

```
return 16;
```



```
li v0, 16
j  _return
```

```
x = 16;
```



```
li t0, 16
sw t0, 4(sp)
```

if it's a local, or...

```
li t0, 16
sw t0, x
```

if it's a global!

the same expression **16** is translated into different code depending on how it's used.

if this seems complicated,
yeah, it is

but we'll come back to this and solve it by being lazy!

Codegen for statements

- lots of statements do **control flow,** meaning the output code is gonna have labels and jumps and branches (ew).
  - fortunately, the templates for these are pretty straightforward and set in stone – there's only one real "right" way to do an if-else.
- sequential statements (like **{** blocks **}**) are no problem.
  - you just translate each statement one after another, and **concatenate** the code together. (yes, seriously!)
- even nested statements are simple thanks to the AST.
  - with the power of recursion, it all works out. *trust recursion*.
- but I think that's enough of an introduction to codegen.
- let's assume we have it working. **does that mean we now have a working program? are we *done with compiler??***
  - well...

Linking and Executables

Abunchafunctions

- a simple program might be one **main()** function and nothing else.
- if we have more functions, we can **concatenate** their code.

```
fn main() {
  f(10);
}

fn g(x) {
  println(x);
}

fn f(x) {
  g(x + 5);
}
```

```
00: li   a0, 10
04: jal  f
08: li   v0, 10
0C: syscall

10: li   v0, 1
14: syscall
18: jr   ra

1C: sub  sp, sp, 4
20: sw   ra, 0(sp)
24: add  a0, a0, 5
28: jal  g
2C: lw   ra, 0(sp)
30: add  sp, sp, 4
34: jr   ra
```

what addresses should the **jal**s jump to?

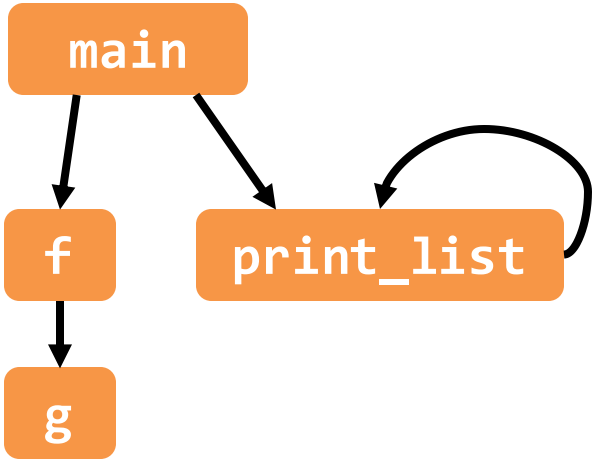
but *when* do we know the addresses of the functions?

what if I define the functions in a **different order?**

what if I call functions **from the stdlib?**

Trying to do too much at once

- really, we're moving past what the compiler should be doing...



a program's **call graph** can be complex, with cycles, multiple dependencies, etc.

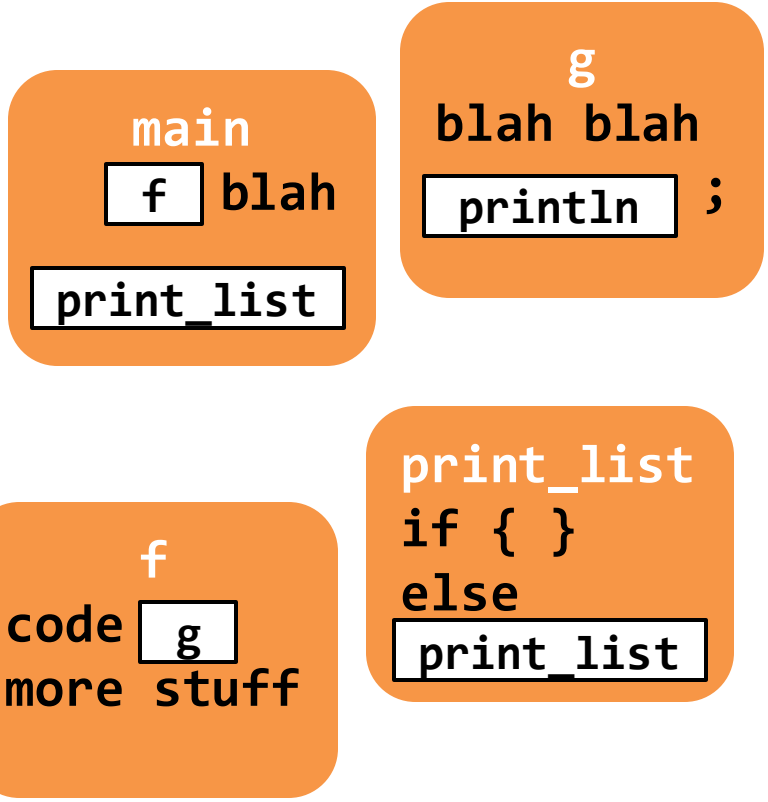
we have to **serialize** this graph when converting it to an executable form.

this process is called **linking.**

rather than the compiler producing a whole program, we have it produce incomplete **fragments,** and let the linker finish the job.

Symbolic linking

- the dependencies between fragments are indicated **symbolically:**
- instead of referring to them by **address,** we do it by **name.**



each fragment has "blanks" to indicate *what* it references.

the linker serializes the fragments and "fills in the blanks."

it can also include fragments from **other parts of the program** or from **libraries** (like the stdlib).

Relocations

- for a number of reasons, the addresses that your code and data end up at **may not be known until right before it's executed!**
- an executable file can have **relocations:** "blanks" where **absolute addresses** are needed, which are filled in at load-time.

In the executable

```
00: li  a0, 10
04: jal 0   <-- Reloc { addr: 0x0004,
08: li  v0, 10          kind: Jump26,
0C: syscall             target: "g", }
10: li  v0, 1  <-- Symbol { addr: 0x0010,
14: syscall             kind: Func,
18: jr  ra              name: "g", }
```

After loading

```
8000: li  a0, 10
8004: jal 0x8010
8008: li  v0, 10
800C: syscall
8010: li  v0, 1
8014: syscall
8018: jr  ra
```

if this code is loaded at address **0x8000**...

Position-independent code

- compilers often have a flag to output **position-independent code,** which uses different instructions to **never use absolute addresses.**
  - this avoids relocations entirely, making things faster to load, and it avoids **duplicating shared libraries in RAM.**
- to do this, the target ISA must be able to calculate addresses of code and data **based on the PC** ("PC-relative addressing"):

like a *branch*-and-link instruction:

```
bal func
```

and load/store instructions which use the **pc** as the base register:

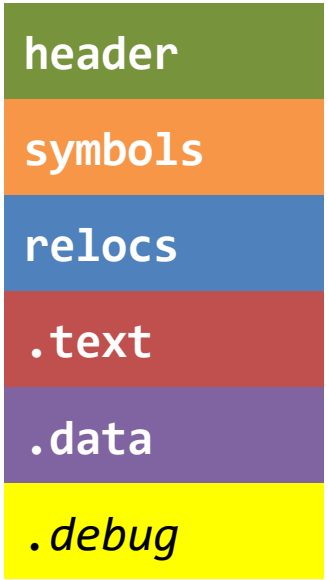
```
lw t0, 0x38C(pc)
```

Debugging info

- converting to the target language is a **lossy** operation.
  - a bunch of info about the source program is lost!
- to make debugging possible, the compiler can also output info like:
  - what **source file and line** each instruction corresponds to
  - the **names** and **locations** of functions, globals, and locals
  - the **types** of storage locations
  - the **structure** of custom types (structs/classes)
  - the arrangement of **stack frames** for each function
- this way, when you run your program in a debugger, you can:
  - step through it **line-by-line**
  - **inspect the contents** of variables, arrays, etc.
  - have the debugger display that stuff in a **human-readable way**
- this info is usually ***HUGE,*** so it's optional and typically removed in "release" versions.

Executable formats

- we can't dump a bunch of instructions out and expect them to run.
- an **executable format** is like a seed, or an..... *egg*
  - it has the code *and* all the metadata needed to support it.
- the OS ABIs define these formats, but they basically all look like this:



a header that identifies the type of the file

a symbol table (names, kinds, addresses)

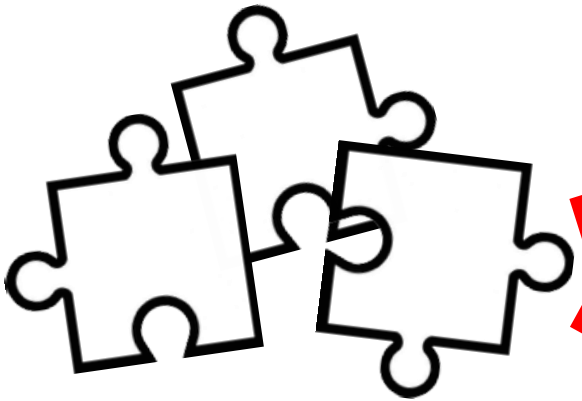
relocation records

and then multiple **sections** for things like code (**`.text`**), globals (**`.data`**), constants (**`.rodata`**), etc.

and optional debug info sections

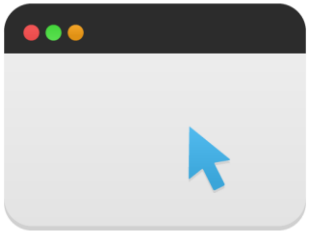
Object files, executables, and libraries

- typically this format is used to represent three kinds of files:



object files: incomplete pieces which might correspond to a single source file or module.

linking them together can produce…



hello.exe

an **executable,** which has an **entry point** (the first function to be run when loaded), or…



hello.dll

a **library** (either static or dynamically linked), which… *doesn't* have an entry point.

instead, it **exports** functions to be used by other files.

So what will *we* do??

- well, this is a compilers course, not a linkers course…
- rather than output *machine* code, our compiler will output **assembly language code. you know, just text.**
  - see, MARS – the MIPS emulator – *has a linker built into it.*
  - so we'll let it "finish the job" and do the linking for us.
- this might sound like a cop-out, but **lots** of compilers work this way!
  - not just toy compilers. ***gcc*** *does this,* if you ask it to.
- many compilers output object files full of machine code so they can skip the assembly step and just go straight to linking…
  - but that's a compilation speed optimization, not a requirement.